

Document Number: P2253R0

Date: 2020-11-15

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [September 9th, 2020](#)
- [September 23rd, 2020](#)
- [October 14th, 2020](#)
- [October 28th, 2020](#)
- [November 11th, 2020](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)

September 9th, 2020

Draft agenda:

- [P2178R1: Misc lexing and string handling improvements](#)
 - Discuss proposal 1: Mandating support for UTF-8 encoded source files in phase 1
- [P2194R0: The character set of C++ source code is Unicode](#)

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Administrative updates:
 - Tom provided an update on the WG14 timeline for C2X.
 - WG14 sent out notification that C2X must be published by August 31st, 2023.
 - That means C2X must be feature complete by August of 2022.
 - Any proposals that we want to get in to C for compatibility reasons needs to be done or (close to done) by August of 2022.
 - PBindels noted that timeline aligns well with C++23.
- [P2178R1: Misc lexing and string handling improvements](#)
 - Proposal 1: Mandating support for UTF-8 encoded source files in phase 1
 - Corentin provided an introduction:
 - The intent of the proposal is to specify that the set of implementation-defined source file encodings shall include UTF-8.
 - This reflects standard practice amongst the major implementations.
 - The mechanism used to specify that the source encoding is UTF-8 remains implementation-defined.
 - Implementations may use any mechanism desired to determine which encoding to use.
 - Quoting Richard Smith: "If we want C++ to be portable, there must be a portable source file encoding".
 - This proposal is orthogonal to any hypothetical proposal to allow differently encoded source files in the same translation unit.
 - Per Unicode guidelines, a UTF-8 BOM would be handled as whitespace.
 - PBrett asked if Corentin is open to follow up papers that tackle additional issues.
 - Corentin responded that Tom has work in-progress that is orthogonal to this paper.
 - PBrett urged adoption; this suffices to compile a source package on any platform.
 - Tom stated that, in practice, there are at least four sets of source files involved when compiling any non-trivial project. Those are, 1) the source files for the

project, 2) the C standard headers, 3) the C++ standard headers, and 4) the platform header files (POSIX, Win32 SDK, etc...). The question is how these disparate projects adopt UTF-8 incrementally.

- Corentin replied that z/OS is an exception that would require support for a mix of UTF-8 and EBCDIC source files; on other platforms, system headers are limited to ASCII in practice. Vcpkg currently compiles all packages as UTF-8.
- Jens stated that standard headers are below the concern of the standard; they are effectively magic and implementations can provide whatever mechanisms they desire to make them work.
- Jens added that it is always ok to restrict oneself to the basic source character set and only use UTF-8 when targeting a UTF-8 supporting compiler.
- Hubert stated that the standard has separate wording for headers vs source files; the latter are also an abstraction in the standard and we can specify that physical source file characters are composed from code units, but that still leaves open the question of what the container is. On many platforms a file is a sequence of bytes, but for some implementations, a source file may be a sequential data set of fixed length records.
- Hubert added that, with regard to specifying use of a UTF-8 BOM to detect the encoding, some implementations have other means for encoding detection; for example, z/OS allows specifying an encoding via filesystem metadata.
- Tom revisited PBrett's scenario of a portable source package on any platform, noted that there are additional source file sets involved if there are third party package dependencies, and stated a desire for a UTF-8 solution to be optimized for deployment and migration across the ecosystem.
- Corentin acknowledged that desire but asserted that is not a goal of the current proposal.
- PBrett asked the attendant core experts how to word this proposal given that the standard doesn't require actual source files.
- Hubert responded that the standard discusses source files but leaves their structure undefined; we can specify a specific form of source file as, e.g., a sequence of UTF-8 code units.
- Corentin agreed and stated that direction matches the intent; a network stream of UTF-8 code units should be acceptable as a source file.
- Jens added that the standard is hazy about what a source file is; it is an abstraction and must not be required to be something that can, for example, be opened by `fopen()`; compilers can be written in any language and therefore can't rely on the C++ notion of files. Specifying a UTF-8 encoding will necessarily require punching through the existing abstraction.
- Zach expanded on Tom's concern and noted that, for existing projects, the compiler already knows how to perform encoding conversions; if we have to alter the specification for translation phases, that seems ok.
- Zach noted that addressing the simple use case where all source files are known to be UTF-8 is important.

- Mark stated that C++20 modules potentially provides additional separation between source files.
- Corentin agreed and emphasized Mark's point.
- Tom responded that exploiting that potential requires the ability to specify encoding options on a per-TU basis, but that is ok; that is an issue for build systems to address.
- Hubert noted that the wording for headers may be quite different than for source files.
- Corentin asked if translation phases 1 through 3 are processed independently for each header.
- Hubert responded that he didn't think we specify that headers (as opposed to source files) are read in this manner.
- PBrett noted that this will require digging a tunnel through the implementation-defined behavior currently present in translation phase 1.
- Corentin agreed, but noted that there is only so much we can specify happen prior to translation phase 1.
- Hubert elaborated on prior comments regarding different wording for headers vs source files; the form of the `#include` directive written with a quoted name is specified to look for a source file and then, if one isn't found, to retry as if the directive were written with a name in angle brackets; headers can be resolved in this form.
- Jens expressed a belief that standard library headers are headers and other things are source files.
- Hubert agreed, but noted that a source file can interpose on a header.
- Tom switched the focus to handling of BOMs and presented a hostile example of not specifying behavior when a BOM is present; one implementation could choose to require a BOM, another could choose not to permit one, and another could choose to allow them optionally and use their presence to inform encoding.
- Corentin replied that, in Unicode, BOMs are not whitespace and should be ignored; they can be used to detect the encoding, but not to reject a code unit stream assumed to be UTF-8.
- Hubert stated that wording is definitely required to express that.
- PBrett stated that a BOM can only appear at the start of a source file; a BOM code unit sequence at the start of a string literal is not a BOM.
- Hubert responded that there may not be agreement on that; there could be special cases for raw-string literals.
- Corentin asserted that a BOM is a non-breaking white space; U+FEFF is "ZERO WIDTH NO-BREAK SPACE".
- PBindels provided a linke to https://www.unicode.org/faq/utf_bom.html#bom6 which states:

Q: What should I do with U+FEFF in the middle of a file?

A: In the absence of a protocol supporting its use as a BOM and when not at the beginning of a text stream, U+FEFF should normally not occur. For backwards compatibility it should be treated as ZERO WIDTH NON-BREAKING SPACE (ZWNBS), and is then part of the content of the file or string. The use of U+2060 WORD JOINER is strongly preferred over ZWNBS for expressing word joining semantics since it cannot be confused with a BOM. When designing a markup language or data protocol, the use of U+FEFF can be restricted to that of Byte Order Mark. In that case, any U+FEFF occurring in the middle of a file can be treated as an unsupported character.

- PBindels noted that the old use of U+FEFF as a zero-width non-breaking space character was deprecated in Unicode 3.
- Tom replied that U+FEFF is only white space when present somewhere other than the beginning of the input; it should be ignored when present as the first code unit sequence.
- Hubert stated that there is a distinction from a source code column perspective, but that there is nothing in C or C++ that requires a token to appear at the start of a line.
- Hubert clarified that there are cases in C++ where adding a space matters.
- PBrett suggested that handling of BOMs be a subject of further work.
- Tom explained that gcc and Visual C++ conflict with regard to handling of BOMs. Gcc will ignore one when directed to compile as UTF-8, but will emit an error otherwise. Visual C++ uses a BOM to inform encoding.
- Hubert raised a question regarding whether a BOM is or is not part of the source file content.
- Jens restated Hubert's question in more concrete terms by asking if a BOM is visible during translation phases 1 and 2.
- Corentin replied that standard practice is inconsistent because tools are not consistent; if we don't want to break existing tools then we can't require a BOM.
- Tom agreed and asserted that no one has suggested a BOM should be required.
- PBrett summarized recent discussion; there is implementation divergence regarding whether a BOM is honored as indicating an encoding vs being ignored.
- PBrett suggested a survey of existing tools is needed.
- Hubert noted that, with respect to Corentin's last statement; we haven't taken a position. It is likely not controversial to ignore a BOM when processing as UTF-8; but we know we don't want to require a BOM.
- Hubert added that it sounds like gcc doesn't use a BOM for encoding detection; in which case the BOM is not a BOM. It sounds like existing compilers effectively ignore it.

- Tom stated that he doesn't know of any experiments that can reveal whether a BOM is handled as white space or removed as file content.
- PBindels asked if that is observable.
- Hubert responded that it is via compiler diagnostics.
- Zach stated that he prefers the approach of a source annotation or command line option to select encoding as BOMs are kind of magical.
- Zach suggested tabling further discussion of BOM handling until/unless we have a separate proposal.
- Corentin observed that current web browsers will prioritize source encoding tags over a BOM.
- PBrett expressed support for not specifying any BOM behavior for an initial proposal.
- Hubert asserted that something must be specified regarding BOM allowance in order for wording to not otherwise reject source files with a BOM.
- **Poll: All implementations should be required to provide an implementaion-defined mechanism to support the scenario in which all source files used within a translation unit are UTF-8 encoded whether or not they have a UTF-8 BOM.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- Tom asked if we should poll whether files must consistently have a BOM.
- Zach asked if that isn't already covered by separate processing of translation phases 1 through 3.
- Jens replied that it is.
- Zach stated that we should not do that poll then.
- Tom agreed.
- **Poll: It should be implementation-defined whether a UTF-8 BOM is used to inform the encoding of a source file.**
 - Mark clarified that voting in favor is a vote for implementation divergence.
 - **Attendees: 10**
 - **SF F N A SA**
4 3 1 2 0
 - **Consensus is in favor.**
 - A: I would prefer well-defined behavior over implementation-defined behavior.
 - Hubert responded that implementation-defined behavior is needed for z/OS in order for filesystem based meta-data to be consulted; requiring 100% conformance with a BOM would be problematic.
- **Poll: The presence or absence of a BOM is a reasonable portable mechanism for detecting UTF-8 source file encoding.**
 - **Attendees: 10**
 - **SF F N A SA**
0 1 0 3 6

- **No consensus; or rather, consensus is that a BOM is not a reasonable portable mechanism for detection of source file encoding.**
 - PBrett explained that his position is weakly held because there may be obscure implementation circumstances where only an unreasonable mechanism exists.
 - Hubert noted that programmers can add a BOM themselves.
 - F: BOMs are used within the Microsoft ecosystem to inform encoding and appear to be useful there.
 - Hubert responded that such a scenario is reasonable for Windows, but that doesn't suffice to claim it as a reasonable portable mechanism.
 - Mark noted that the first poll taken leaves this option available.
 - Corentin stated that the source annotation approach is a superior solution.
- Tom stated that the next meeting will be in two weeks, on September 23rd, and will focus on [P2194](#).
 - Tom asked Jens to confirm that he has a competing paper.
 - Jens responded affirmatively, but that he is waiting for [P2029](#) to land.
 - Jens reminded the group that there is need to progress [P1949](#); it appears to be stuck in EWG.
 - Tom asked Steve if [P1949](#) was ready for another round in EWG.
 - Steve confirmed that it is, has been submitted for the mailing, and that he will prepare slides.
 - Tom promised to ping JF.
 - *[Editor's note: Tom did so and JF put it on the EWG schedule for Thursday, September 24th.]*
 - Hubert reminded the group that there will be a plenary in November and that papers made tentatively ready by EWG will require another meeting to be approved.

September 23rd, 2020

Draft agenda:

- [P1949R6: C++ Identifier Syntax using Unicode Standard Annex 31](#)
 - Ensure we are collectively prepared for presentation to EWG on Thursday.
- [P2194R0: The character set of C++ source code is Unicode](#)
- Review [Boost.Text](#) changes following initial Boost review.

Attendees:

- Corentin Jabot

- Hubert Tong
- Jens Maurer
- Mark Zeren
- Martinho Fernandes
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P1949R6: C++ Identifier Syntax using Unicode Standard Annex 31](#)
 - Tom explained that the goal of this review is to ensure we are collectively prepared for the presentation to EWG on Thursday. Specifically:
 - To perform a run-through of Steve's slides and provide feedback.
 - To play devil's advocate in anticipation of questions or concerns that may be raised at the EWG telecon.
 - Steve presented:
 - *[Editor's note: Steve's draft slides are available in the SG16 mailing list archive at <https://lists.isocpp.org/sg16/2020/09/1866.php>.]*
 - The slides present a brief summary of the proposal, challenges with emoji support, script impact, uses of [UAX #31](#) by other languages, and wording overview.
 - Summary slides present the identifier syntax, requirements for [NFC normalization](#), and that this proposal addresses C++20 NB comment NL029.
 - The status quo is that identifiers can be surprising as they may look like symbols or incorporate characters that look like operators.
 - The proposed identifiers are closed over normalization and are guaranteed stable by the Unicode standard.
 - The status quo for emoji support in identifiers is that it is accidental, incomplete, and broken.
 - Emoji are not guaranteed stable by the Unicode standard. Supporting emoji could introduce instability to identifiers over time; what was once an identifier may cease to be one in the future.
 - Some characters categorized as emoji are surprising; for example, #, *, and the decimal digits 0 through 9 are all categorized as emoji since they may begin an emoji keycap sequence.
 - Supporting emoji would require being inventive; there is no standard for use of emojis in identifiers.
 - Support for emoji in identifiers could be added later with an appropriate proposal.
 - The proposal does not exclude any scripts.

- Some scripts, including English, contain words that are not valid as identifiers. English examples include `can't`, `won't`, and `mother-in-law`. Likewise, some scripts require use of invisible characters like ZWJ (U+200D) and ZWNJ (U+200C) to spell some words and, in some cases, these characters are all that differentiate some words. For example, the Farsi words `نامه‌ای` (U+0646 U+0627 U+0645 U+0647 U+0627 U+06CC) and `نامه‌ای` (U+0646 U+0627 U+0645 U+0647 U+200C U+0627 U+06CC) differ only by the presence of a ZWNJ.
- Other languages, including at least Java, Python, Erlang, Rust, and ECMAScript have adopted UAX #31.
- Wording has been provided by a CWG expert.
- Jens suggested that the "OTHER "WEIRD IDENTIFIER CODE POINTS"" slide be updated to make it clear that the content reflects the C++20 status quo.
- Zach suggested being more specific about the end result of allowing unassigned code points in identifiers; that choice enabled some emoji to be incorporated in an unprincipled fashion.
- Jens suggested increasing the font size for examples.
- PBrett requested updates to slides with examples to make it clear whether they reflect the C++20 status quo or proposed behavior.
- Jens questioned the motivation behind some of the presented examples; if the challenge faced by supporting emoji is algorithmic complexity, then it would make sense to present the complicated examples first.
- Zach suggested that it might be productive to include the grapheme cluster rules on a slide.
- Steve responded that the paper includes a complicated regular expression copied from [UTS #51](#) that can be used to match a possible, but not necessarily valid, emoji sequence; that could be added.
- Tom commented on the "SOME SURPRISING THINGS ARE EMOJI" slide; that example demonstrates that lexing would be made more challenging because emoji sequences can begin with members of the basic source character set.
- Steve agreed and presented a related concern; whether `1` followed by U+20E3 (COMBINING ENCLOSING KEYCAP), a valid emoji, would be allowed to start an identifier.
- PBrett noted that a standard that addressed how to incorporate emoji support into identifiers would benefit other languages.
- Jens requested the use of lowercase letters in the examples of English words that can't be used as identifiers in order to maintain focus on the punctuation characters.
- Tom suggested that the "ZWJ AND ZWNJ" slide make it more clear what is being illustrated; that `نامه‌ای` is a valid identifier, but that `نامه‌ای` is not because it includes a ZWNJ.
- Hubert suggested updating spacing to make the ZWNJ presence more clear.
- Hubert noted that UAX #31 doesn't prescribe handling of ZWJ and ZWNJ, but rather provides a recommendation; it would be disingenuous to claim disallowance of

these characters based on UAX #31.

- Steve acknowledged and noted that script analysis could be performed, but that doing so would be difficult.
- Corentin asserted that we are not qualified to make decisions that affect, for example, Farsi; such decisions should be driven by domain experts.
- Martinho requested adding rationale for why ZWJ and ZWNJ characters should be rejected in identifiers.
- Hubert responded that the reason for rejection is that their presence may not affect presentation.
- Corentin noticed Rust listed on the "OTHER ADOPTERS" page and stated that Rust hasn't adopted UAX #31 yet.
- PBrett expressed a belief that Rust had adopted UAX #31.
- Martinho provided a link to the Rust tracking issue, <https://github.com/rust-lang/rust/issues/55467>, and explained that, though the issue is not yet resolved, the proposal has been accepted and is implemented in the development tree; there just hasn't been a new release of Rust that includes it yet.
- Tom suggested that C# could be added to the list of adopters.
- Corentin disagreed and noted that the C# specification is still based on category properties, not the XID properties.
- Tom opined that it sounds like C# was an early adopter of UAX #31 and still uses the older properties; C# presumably has identifier stability issues as a result.
- Corentin suggested that the wording slide isn't helpful and could be dropped.
- Jens requested that, if the wording slide is retained, that the text be left-align.
- Jens noted that there is a formal step that EWG affirm wording, but that it isn't necessary to dive into details.
- Jens recollected that, during the last EWG review, the major concerns were about emoji and script support.
- Tom and Steve both confirmed that recollection.
- Zach commented that the script restrictions involving Farsi was just an example; the point is that some scripts have similar limitations as English with respect to some words not being valid identifiers. Steve agreed and noted that some words in some scripts require white space.
- Jens added that we know of no script that is completely excluded.
- PBrett asked if a poll to forward the paper to CWG should be expected in the EWG telecon.
- Tom and Jens described the tentatively ready process adopted in Prague; that the paper can be made tentatively read at the next plenary, and then adopted at the following plenary.
- Tom asked if any other languages are known to support emoji in identifiers.
- Corentin replied that Swift does.
- Tom responded that Swift currently uses the same approach that C++20 does, so Swift's emoji support is just as broken as that in C++20.
- Corentin added that Swift also allows some emoji as operators.

- Corentin opined that emoji are generally considered like symbols and therefore shouldn't appear in identifiers.
- Martinho noted that CSS allows just about any character in an identifier.
- Tom discussed an additional complication faced by supporting emoji; the text and emoji presentation styles. Emoji characters have a default presentation style that can be changed by a presentation selector; the question is whether an emoji sequence with a presentation selector that matches the default presentation style and an emoji sequence without a presentation selector should be considered valid spellings of the same identifier.
- Tom noted that the slides don't present what would be required in order to support emoji well.
- Tom asked if there is a fast path option such that the complexity needed to support emoji is only paid if emoji are used.
- Corentin replied that lexing behavior would have to become EGC based.
- Tom asked how much of the Unicode property DB would be required for emoji support and noted that the emoji data text files are about 67K.
- Corentin noted that other data may be required depending on design decisions.
- [P2194R0: The character set of C++ source code is Unicode](#)
 - PBrett presented, assisted by Corentin.
 - This paper forked from proposal 9 of [P2178R1: Misc lexing and string handling improvements](#).
 - SG16 has had several passionate discussions about this in the past.
 - The ideas presented are Corentin's, PBrett provided the prose.
 - Key points:
 - This is not a proposal to change the standard.
 - This is not a proposal to change any implementations.
 - This is about how we think about lexing and parsing.
 - C++20 is, perhaps accidentally, correct as is by requiring all source input characters to be representable by either basic source characters or Universal Character Names (UCNs).
 - Taking a dependency on Unicode is reasonable.
 - In C++20, translation phase 1 is effectively limited to Unicode scalar values by the requirement that translation phase 1 produce only basic source characters and UCNs.
 - Only UTF encodings produce scalar values without requiring a character set map.
 - *[Editor's note: presentation was cut short at this point by discussion and time constraints.]*
 - Hubert asked why the paper contains proposed wording if it is not intended to change the standard.
 - PBrett responded that the Proposed Wording section will be removed in the next revision and additional content added to clarify the difference between Unicode scalar value and character.

- *[Editor's note: the following discussion concerns the difference between a Unicode scalar value (a Unicode code point that is not a surrogate code point) and a Unicode assigned character (a Unicode code point that represents an abstract character).]*
- Jens wondered about the ramifications of supporting Unicode scalar values as opposed to assigned characters and when character properties become relevant; the Unicode of 1993 differs substantially from Unicode today.
- Martinho noted that Unicode has not always maintained backwards compatibility.
- Hubert agreed and noted that the Unicode code space shrunk in Unicode 2.0 when UTF-16 and surrogate code points were defined.
- Jens observed that UCNs can be explicitly written to produce arbitrary scalar values, including scalar values corresponding to unassigned code points.
- PBrett responded that explicit UCNs are rarely seen outside of compiler test suites.
- PBrett added that, conceptually, post translation phase 1, only scalar values remain.
- Jens expressed uncertainty; that it isn't clear that newly assigned Unicode characters have meaning for an existing C++ standard.
- PBrett responded that such meaning is immaterial since explicit UCNs are allowed to name unassigned code points.
- Jens acknowledged that we have to define behavior for all possible Unicode scalar values whether assigned or not.
- PBrett agreed and noted that such behavior impacts the set of allowed identifiers as proposed in [P1949: C++ Identifier Syntax using Unicode Standard Annex 31](#).
- Jens wondered if character properties are actually relevant for translation phase 1.
- Hubert stated that it is important to understand whether there would be a benefit to insisting that code points correspond to assigned characters in particular contexts.
- Hubert added that stating that lexing and parsing are in terms of assigned characters would be distracting in general.
- Martinho returned discussion to the example Jens provided of an implementation being behind the current Unicode standard; there are motivating use cases for use of a UCN for a code point that has not yet been assigned by Unicode in a published standard. For example, in anticipation of a new Japanese calendar era, U+32FF (SQUARE ERA NAME REIWA) was reserved before the new era began though the new character did not appear in a published Unicode standard until after the era began.
- Corentin noted that there is no intent to remove support for explicit UCNs.
- Corentin added that, whether a code point is assigned or not only matters during translation phase 1 conversions.
- Hubert asserted that it would be undesirable to specify constraints on translation phase 1 conversions; some implementations use `iconv`, implementors may not want to validate `iconv` and instead document their implementation-defined behavior as, "whatever `iconv` does".
- Corentin stated that there is no intent to restrict implementation-defined character mapping in translation phase 1.

- PBrett responded that, in principle, requiring Unicode characters post translation phase 1 would require rejecting Unicode scalar values corresponding to unassigned code points.
- Jens reflected on earlier terminology discussions and joking about what "character" means because it is hazy and strange.
- Jens opined that, unless "character" is defined in such a way that any benefits it offers over "scalar value" are made apparent, we should avoid it.
- PBrett disagreed.
- Jens noted that the telecon was about to end and stated that it may be useful to expand on that at a future telecon.
- PBrett asked if more time should be dedicated to this topic.
- Tom expressed support for more time as different perspectives suggest we would benefit from increasing understanding.
- Hubert noted that there will be a competing paper partially motivated by a desire for the standard to remain abstract and not tied too heavily to Unicode.
- Hubert noted that the competing paper mentioned at the end of the last telecon is partially motivated by a desire for the standard to remain abstract and not tied too heavily to Unicode.
- *[Editor's note: That paper would align the C++ standard with the C standard model of extended characters. It remains in draft status pending CWG approval of [P2029](#) due to wording dependencies.]*
- Corentin expressed little interest in the distinction between characters and scalar values; that character properties are what matters.
- Corentin added that LEWG will be backed up in C++23, so it is good to focus on these core language issues now.
- Mark agreed; this is fundamental work.
- Tom stated that the next meeting will be in three weeks, on October 14th, and that we'll probably start discussion with a review of recent updates to [Boost.Text](#), and then continue discussion of this paper.

October 14th, 2020

Draft agenda:

- Discuss migration from the Cpplang Slack workspace to another Slack workspace or chat service.
- [Boost.Text](#):
 - Review changes made following the initial Boost review.
- [P2194R0: The character set of C++ source code is Unicode](#):
 - Continue discussion.

Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Nathan Baggs
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Tom asked for a volunteer to review [P1030R4](#) for any new SG16 concerns.
- Steve bravely volunteered to do so.
- PBrett asked if CWG has a substantial backlog like LWG does.
- Jens responded that CWG does not, that it is awaiting tentatively ready papers from EWG, and continues to do issue processing.
- Discuss migration from the Cpplang Slack workspace to another Slack workspace or chat service:
 - Tom introduced the issue. Concerns have been raised about governance of the Cpplang Slack workspace, particularly with regard to enforcing a code of conduct (CoC). Several prominent SG16 members have deactivated their accounts. If conditions don't improve such that those members are comfortable reactivating their accounts, we'll have to migrate elsewhere.
 - Zach confirmed the situation and stated that the financier of the workspace has no interest in moderation; a number of people have reported issues without satisfactory resolution.
 - PBrett raised the option of not using a chat service for SG16 business and indicated difficulty with use of Slack.
 - Steve stated that he is present on Slack regularly, but only monitors a small handful of channels that have so far not attracted problems.
 - Steve added that he is not concerned about migrating elsewhere, but would like to retain an open channel.
 - Steve noted that the SG16 mailing list is open and has not attracted problems.
 - Tom opined that use of a chat service has been helpful to build and maintain cohesiveness among SG16 participants.
 - Jens stated that he has not used Slack, that the mailing list should be the primary means of communication, and that the means of communication should not exclude anyone.
 - PBrett mentioned that the [#include<c++>](#) community are investigating a collaboration system that requires participants to be vouched for.

- PBrett expressed a preference for a WG21 provided service with an actively enforced CoC.
- Corentin suggested that we wait to see if WG21 decides to offer such a service.
- Tom agreed and noted that discussion within WG21 is already happening.
- [Boost.Text](#) updates:
 - Zach provided an overview of the changes inspired by the Boost review:
 - [*Editor's note: The Boost review threads are available at <https://lists.boost.org/Archives/boost/2020/06/249242.php> and <https://lists.boost.org/Archives/boost/2020/08/249594.php>.]*
 - The string layer was removed.
 - Many of the previously concrete types are now templates. For example, `text` is now an alias of a `basic_text` specialization analogous to `std::string` and `std::basic_string`.
 - `basic_text` is parameterized by Unicode normalization, code unit type, and string container.
 - `text` and `text_view` are NFC, UTF-8, use `char` as the code unit type, and use `basic_string<char>` as the string container.
 - More type deduction is done now.
 - Various usability improvements were made.
 - Interfaces are now constrained with C++ Concepts when compiling as C++20.
 - A new algorithm was added to perform normalization during insert and erase operations
 - `basic_text` is an adapter over a string container; this avoids allocator awareness while still allowing use of allocators.
 - The [stream-safe text format](#) is now enforced. This may result in truncation of extended grapheme clusters (EGCs), but this is reasonable as there is no technical reason for EGCs longer than the stream-safe text format length.
 - Tom asked how normalization on insert and erase works.
 - Zach responded that there is an algorithm to find the previous and next stable code points and then normalize in between.
 - PBrett asked about the stream-safe text format and whether it can lead to loss of information.
 - Zach responded that, yes, it can lead to silent loss of information, but only in unrealistic scenarios; real text has no need for an EGC to be longer than 30 scalar values. The maximum EGC length for the stream-safe text format was selected to enable use of statically sized 128 byte buffers. The longest known sequence of scalar values needed for a real character is 18, but sequences of length 4 are more common in actual text. The stream-safe text format is part of the Unicode standard.
 - PBrett asked about emoji compositions.
 - Zach replied that all defined emoji sequences are limited to 7 or 8 scalar values.
 - Steve asked about the use of C++ Concepts for range insert and join operations, what constraints are placed on the operands, and whether bidirectional iterators are required or whether forward iterators suffice.

- Zach replied that he tried to require only forward iterators, but concluded that bidirectional or better is required for efficiency as maintaining normalization requires look back.
- Tom noted that maintaining normalization may require mutating scalar values on either side of the operation as well.
- Corentin returned to the stream-safe text format topic and stated his understanding that emoji sequence can get longer than the 7 or 8 scalar values that Zack mentioned, but not longer than 18. [Zalgo](#) is impacted, but that is ok. Not using the stream-safe text format would be very expensive.
- Zach added that the original purpose of the stream-safe text format was to read a buffer, normalize it, then move on; normalization occurs one EGC at a time.
- Victor noted that one of the changes was to support encodings other than UTF-8 and asked if the default would be implementation-defined.
- Zach replied that the parameterization is similar to `std::basic_string`. The default is NFC, UTF-8, and storage via `std::basic_string<char>`.
- Jens stated that a UTF-8 string literal yields an array of `char8_t` and that `std::string` is a volabulary type that appears in API boundaries; this interface may be unfriendly.
- Zach replied that he was unconcerned about that mismatch.
- Tom noted that `std::string` can not be expected to always hold UTF-8.
- Jens expressed a desire to avoid use of `reinterpret_cast`; adopting `char8_t` may have been a mistake, or is something that everything has to adapt to.
- Steve stated that he sees UTF-8 data transported in `std::string` frequently and problems occur when programmers fail to track encoding; this doesn't make the situation worse.
- Corentin opined that our direction should be that `char` is used for the system encoding and `char8_t` is used for UTF-8.
- Corentin added that a magic compile-time function may be needed to convert `char8_t` literals to `char`.
- Tom asked about the use of separate template parameters for encoding vs normalization and whether those could be combined so that normalization is a property of the encoding.
- Zach responded that the encoding is deduced by the size of the code unit type.
- Tom asked if Zack considered a single template parameter that specifies encoding with associated normalization.
- Zach responded that he did not; that the expectation is that users will want to change just the normalization form.
- PBrett asked what additional feedback Zack would like from SG16.
- Zach requested participation in the next Boost review and that other feedback is welcome, particularly with regard to defaults.
- Zack added that any questions posed will be presented at the start of the next Boost review.
- Steve asked if the current code is aligned with these changes.

- Zach responded yes, the [code on github](#) is up to date.
- [P2194R0: The character set of C++ source code is Unicode](#):
 - PBrett presented:
 - The principle advantage of using Unicode to describe C++ lexing and parsing is that it is the only system that can do so comprehensively.
 - Unicode defines what a blank space character is and means.
 - Use of Unicode to describe the standard does not impose a requirement that Unicode be used as the internal character set for implementations.
 - Universal-character-name (UCN) reversion does not require support for non-Unicode characters. For example, an implementation that uses UTF-32 as the internal character set could use an unused bit to track characters that require UCN reversion.
 - Trigraphs are an alternative way to express a basic source character and are supported by exploiting implementation-defined behavior in translation phase 1.
 - Tom noted two concerns that he would like to ensure the proposal addresses:
 - That the described conversion accurately reflects existing practice.
 - The core wording issue with reversion of UCNs; that there is no lexical element to revert to.
 - Jens stated that, from a core perspective, the UCN reversion is well recognized as hand waving.
 - Jens noted that proper support for Unicode source files is still relatively new in gcc.
 - Jens added that migration to a different character model that better specifies behavior, perhaps in terms of Unicode code points would be an improvement.
 - Corentin stated that he and Peter Brett will bring a paper proposing to remove UCN introduction during translation phase 1.
 - Hubert stated that handling source characters as Unicode code points is a transition closer to the C model.
 - Hubert suggested that unmapped characters may, perhaps, require special handling in comments.
 - Hubert added that the UCN reversion is magic and that it effectively matches the extended character model in a restricted form; implementations use an extended character model.
 - PBrett expressed uncertainty as to how trigraphs, which are no longer specified in the C++ standard, get involved here.
 - Jens stated that translation phase 1 being implementation-defined was the compromise escape hatch that allows implementations to continue to support trigraphs.
 - Tom mentioned that concerns about trigraphs may be due to his asking how they are affected by UCN reversion.
 - PBrett noted that such concerns are effectively identical to cases of the same abstract character being represented by potentially different code points or code point sequences in the source code as can happen with rare cases in Shift-JIS.

- PBrett added that he and Corentin feel strongly that the source character set should not be observable by portable programs.
- Hubert stated that no tax should be imposed on either the programmer or the implementation if the source character set and execution character set are the same; an implementation should not have to perform character validation in such cases.
- PBrett acknowledged and noted that the standard should not forbid implementations to preserve values through translation.
- Corentin stated that there are characters that are not representable in Unicode, but such characters tend to be ones that are not represented on computers at all, or not used in source files.
- Jens stated that the relative effect on wording for a Unicode code points vs extended character model is expected to be minimal.
- Tom asked Jens if the character model he has in mind is incompatible with what Peter and Corentin are proposing.
- Jens responded that the distinction is effectively Unicode vs Unicode+X. But since unassigned code points can be specified in UCNs, we're effectively going to end up with a Unicode+X model regardless.
- Tom stated that the next meeting will be October 28th and that we'll continue discussion of P2194.

October 28th, 2020

Draft agenda:

- [P2194R0: The character set of C++ source code is Unicode](#)
 - Continue discussion.
- [P1885R3: Naming Text Encodings to Demystify Them](#)
 - Review updates since the [review of D1885R2 in Prague](#).

Attendees:

- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Tom provided an update on the possibility of a WG21 hosted chat service.
 - Discussion at the recent pre-meeting administrative telecon ended with an action item for Tom to submit a paper to be reviewed and discussed at a to-be-scheduled administrative telecon.
- Tom announced that we'll discuss [P2093: Formatted Output](#) at our next telecon assuming we complete review of [P1885R3: Naming Text Encodings to Demystify Them](#).
- [P2194R0: The character set of C++ source code is Unicode](#):
 - PBrett introduced:
 - Continuing discussion from prior telecons; we need to follow up with regard to how important it is that all inputs be mappable to Unicode scalar values.
 - This is not a proposal to require use of Unicode scalar values as the internal encoding; rather, that the observable behavior be as-if the implementation used a Unicode encoding internally. This approach eases specification.
 - Arguments of "someone might do some crazy thing" are not persuasive.
 - Hubert suggested there isn't actually significant opposition to this view; it was pointed out during the last meeting that the wording differences would be minimal.
 - Hubert added that the key difference is the addition of a restriction; that all characters be mappable through the Unicode code space.
 - PBrett agreed and noted that much of the current specification is already described in terms of Unicode.
 - Tom asked to what degree existing implementations reflect adherence to a Unicode-only model.
 - PBrett responded that the paper's "Current implementation practice" section notes that most existing implementations use UTF-8 as the internal encoding, but that IBM's xLC for z/OS and EDG are exceptions that use EBCDIC and ASCII based code pages instead.
 - PBrett added that Clang on z/OS uses UTF-8 as the internal encoding and converts source files from EBCDIC when they are read.
 - Hubert clarified that Clang on z/OS only supports the IBM-1047 character set.
 - Hubert added that there has been consideration for use of `iconv` to convert source files, but there are concerns about encoding names not being portable.
 - PBrett added that gcc can also be built to use UTF-EBCDIC as its internal encoding.
 - PBrett returned to Tom's original question and stated that EDG and IBM's xLC are the only implementations that do not use a Unicode encoding as the internal encoding, but the difference is not observable.
 - Tom asked if a universal-character-name (UCN) that names an unassigned character still constitutes a Unicode scalar value.
 - Jens responded that it does; Unicode scalar values include all valid Unicode code points except for the surrogate code points used for UTF-16.
 - Tom stated that, at our last telecon, Hubert asserted that implementations should not be required to diagnose invalid characters in string literals, at least not when the source file encoding matches the execution encoding.
 - Jens noted that such a diagnostic only requires a simple range check.

- PBrett brought up prior discussions regarding source files containing ill-formed code unit sequences.
- Tom opined that that concern does not fall under the scope of this paper.
- PBrett replied that it is relevant for the scenario where ill-formed code unit sequences are present in string literals and where the implementation copies the code units.
- PBrett added that this only seems to occur in mojibake scenarios; when compiling ISO-8859-1 encoded source files as UTF-8 for example.
- Steve stated that it is preferable to use explicit escape sequences when ill-formed code unit sequences are actually desired than to rely on the implementation failing to diagnose such sequences.
- PBrett agreed and noted that such cases are now being found in projects he works on due to increased use of tools that expect UTF-8 and diagnose an ISO-8859-1 encoded copyright symbol as an ill-formed code unit sequence.
- PBrett added that these projects are migrating source files to UTF-8 in response.
- Tom asked where we stand as a group on this paper.
- Jens provided a summary of the three UCN models described in the C99 rationale. WG21 chose model A for C++, WG14 chose model B for C.
- [*Editor's note: The referenced ["C99 rationale" document](#), in section 5.2.1, subsection "UCN models", states:*

Once this was adopted, there was still one problem, how to specify UCNs in the Standard. Both the C and C++ committees studied this situation and the available solutions, and drafted three models:

- A. Convert everything to UCNs in basic source characters as soon as possible, that is, in translation phase 1.
- B. Use native encodings where possible, UCNs otherwise.
- C. Convert everything to wide characters as soon as possible using an internal encoding that encompasses the entire source character set and all UCNs.

Furthermore, in any place where a program could tell which model was being used, the standard should try to label those corner cases as undefined behavior.

]

- Jens stated that, in the C++ model, after translation phase 1, all source input is described in terms of the basic source character set; this model is supposed to be indistinguishable from the other models, but is not because only Unicode scalar values may be representable. This is arguably a defect of the model.
- Jens noted that the other models allow extended characters.
- Jens expressed agreement with moving away from the current model as it produces more specification issues, at least in part due to confusion with explicit UCNs, than a model based on code points or scalar values would.
- PBrett noted that implementations convert explicit UCNs lazily.
- Jens expressed support for requiring that compilers be able to represent all Unicode scalar values with support for extended characters; this would suffice to avoid cases of accidental UCN formation by token pasting.

- Jens expressed skepticism regarding being able to meaningfully prohibit edge cases like distinct Shift-JIS characters that map to the same Unicode character, but this could be acknowledged.
- Hubert stated that Unicode suffices for round tripping of almost all characters; exceptions are mostly theoretical. An implementation can map such characters to code points in the Unicode Private Use Area (PUA) to support round-tripping of such special cases if desired; this is a QoI issue.
- Hubert added that another choice would be to make such cases explicitly ill-formed thereby requiring a diagnostic warning of non-portable code.
- PBrett opined that the standard should not normatively acknowledge a Unicode+X model; implementations can still offer extensions.
- Jens asked Hubert, as an authoritative source of requirements for EBCDIC-based implementations, whether a Unicode+X model in the standard is required, perhaps for control characters that are not semantics preserving in Unicode.
- Hubert replied that the status quo is that semantics in terms of how the character is written are not preserved.
- Steve noted that some programmer somewhere will therefore be relying on that.
- Jens asked Hubert if a Unicode-only model is a concern for his implementations.
- Hubert replied that he does not believe it to be problematic; there is a mapping to Unicode from almost every character and for any special cases, there is an escape hatch via translation phase 1.
- Tom stated that he has now been persuaded that a Unicode+X model is not necessary for legacy code and that a diagnostic for non-portable code is desirable.
- PBrett stated that the unobservability of the encoding of the source code is an important principle.
- Mark reflected on the past two years and how our thoughts regarding how to write simpler libraries have lead us to this conclusion.
- Steve added that our prior discussions regarding character and string literals encountered challenges with the translation phases; the current wording made it challenging to talk about such concerns.
- Jens stated that the last time we took away latitude in the standard was when we standardized on 2s-complement; research conducted found that there were no modern machines that were not 2s-complement.
- Jens added that, assuming we have sufficiently scrutinized the concerns here, and we have Hubert here to argue for one of the outlier platforms, it would still be nice to have more input from other implementors.
- PBrett responded that there is no intention to remove behavioral latitude here.
- Jens stated that he finds the mapping of control characters to something that isn't semantics preserving in EBCDIC concerning, but if Hubert is ok with it, then he trusts that it is not a problem in practice.
- Hubert responded that the wording presented continues to allow both models.
- PBrett elaborated; there exists a mapping from EBCDIC to Unicode and it is indistinguishable whether the input was mapped to Unicode and then magically

reverted.

- Hubert noted that this model does not permit different behavior for a physical character vs use of a UCN.
- Tom replied that, per previous discussion, an implementation can differentiate behavior there as if it had mapped the character through the PUA; a programmer could encode a PUA character using a UCN.
- PBrett added that an escape sequence could also be used.
- Hubert stated that previous discussion raised concerns that had rather weak arguments and suggested that the paper could be revised to deemphasize those concerns.
- PBrett responded that this is a defensive paper written with the goal of defending the status quo.
- Tom stated that his impression had been that this paper would move in the direction of removing the introduction of implicit UCNs in translation phase 1.
- PBrett replied that that would be a different paper; perhaps the one Jens has in mind.
- Jens stated that that paper still awaits [P2029](#) adoption in the standard.
- Hubert suggested a goal; we want to affirm this paper as a way of ensuring the portability of source code across environments.
- Hubert added that characters that don't map to Unicode are mostly theoretical and not of practical significance.
- Hubert stated that the last point we need to address is preservation of the potentially non-semantics preserving behavior of translation phase 1.
- PBrett countered that he and Corentin want to ensure that, if a source file is encoded as UTF-8, that semantics are preserved.
- PBrett added that there isn't really a future for this paper; The next direction would be a paper to remove implicit introduction of UCNs.
- Jens noted that the status quo is that the standard does not directly support characters outside of Unicode today and that it would be helpful to reaffirm that the status quo is sufficient for EBCDIC; that would leave us just needing motivation sufficient for changing the translation description in the standard.
- Jens added that he would like to have a poll that we are happy to transition to a model specified in terms of Unicode scalar values.
- Hubert expressed support for that poll.
- Tom asked for a summary of what we consider to be the motivation for changing the model in the standard.
- Jens replied that most implementations use the extended character model internally; switching from the UCN model to the Unicode scalar value model better reflects implementation practice and avoids some of the specification issues that arise with the UCN model.
- PBrett asked if this change would be evolutionary given that no implementations would be affected.
- Jens replied that it may not be, but that EWG would likely prefer to discuss it.

- Hubert noted that there may be implementation impact because the move away from implicit UCNs may make some behavior well-defined that is currently undefined behavior.
- **Poll: The model of description in the C++ language standard should be switched from basic source character set + UCNs to Unicode scalar values.**
 - Mark clarified that a vote in favor is a vote for implementation divergence.
 - **Attendees: 8**
 - **SF F N A SA**
5 2 1 0 0
 - Consensus is in favor.
- Hubert raised a question of when explicitly written UCNs are translated.
- Jens acknowledged that as a concern; we'll need to determine what the status quo is and what behavior is desired.
- Hubert added that we'll need to determine whether it makes sense to preserve undefined behavior in existing cases.
- Tom stated that the next meeting will be November 11th.

November 11th, 2020

Draft agenda:

- [P1885R3: Naming Text Encodings to Demystify Them](#):
 - Review updates since the [review of D1885R2 in Prague](#).
- [P2093R2: Formatted output](#)

Attendees:

- Corentin Jabot
- Nathan Baggs
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Tom announced that Peter Brett has agreed to take the role of assistant chair for SG16.
 - Herb requested that all WG and SG chairs nominate someone to act as an assistant chair.

- Peter Brett volunteered to do so.
- The new roles have not been communicated to all of WG21 yet.
- Should anyone have any concerns or questions, please let Tom know.
- Thank you Peter!
- [P1885R3: Naming Text Encodings to Demystify Them](#):
 - Tom provided a brief introduction:
 - We last looked at this paper in Prague and approved it pending some additional research that Corentin agreed to do.
 - Corentin completed that research and the updates are present in R3.
 - The purpose of today's review is to review those updates; not to revisit the proposed design.
 - Corentin provided a summary of the updates.
 - Lists of encodings supported by several text facilities and other specifications were added.
 - These lists identify some encodings that are not registered with the IANA registry.
 - Lists of known encodings not registered with the IANA registry are present in annex B.
 - No other registry is more comprehensive than the IANA registry.
 - The [WhatWG Encoding standard](#) contains a [list of encodings](#) that continue to be encountered by browser and non-browser user agents on the web; each of these encodings is registered with the IANA DB.
 - PBrett asked if the primary use case for the paper is detection of the literal and system encodings.
 - Corentin confirmed that is the primary motivating use case.
 - PBrett noted that the proposal goes to a fair amount of effort to be comprehensive in this respect.
 - Corentin acknowledged and stated that the proposal is motivated by requirements to cope with legacy environments; if a magic wand could be waved to make those requirements disappear, then waving it would be a better use of our time.
 - Corentin added that SG16 is confident that legacy encodings will continue to be relevant for many years.
 - Corentin summarized the goals of providing a comprehensive list of encodings:
 - To ensure portability across implementations.
 - To avoid WG21 having to determine which legacy encodings are and are not important.
 - Steve agreed with the primary goals as stated by Peter and Corentin, but added that this functionality would be useful for general I/O interfaces.
 - Steve provided an example; he works on a product that integrates with email services and has to deal with email that is encoded in a variety of legacy encodings in addition to UTF-8.
 - Steve noted that they could limit support to only those encodings that are on the WhatWG list.

- Steve added that the WhatWG list is a closed list and that it will not be extended.
- Corentin noted that the WhatWG list is a subset of the IANA list, but the reverse is not.
- Corentin added that the IANA registry may be extended later, though it has not been updated since 2004.
- Tom stated that another potential use case is for JeanHeyd's transcoding work.
- Jens noted that the proposed feature does not expose which encodings are and are not in the WhatWG encoding standard.
- Jens suggested that, if there are uncertainties with regard to fit for other uses, that the design could be scaled down so that it only exposes the literal and system encodings. When standardizing an interface for dealing with endianness, a minimal approach was taken and that side-stepped design discussions.
- Corentin asked Steve if he would like to have an interface that exposes which encodings are and are not included in the WhatWG encoding standard.
- Steve replied that he has a need to be able to communicate and set expectations and added that it would not be terrible if the ability to consume additional encodings was added accidentally, but that there is a need to ensure output is not produced in encodings other than UTF-8 and UTF-16.
- Steve added that he doesn't know if he has a good use case for being able to query which encodings are blessed by the WhatWG encoding standard and expressed uncertainty whether such programmatic support would be useful.
- Corentin stated that the design is the way it is so that encodings that are not known to a given system can still be named.
- Corentin expressed a preference for waiting to scale the feature down pending feedback from LEWG.
- Corentin noted that he still has feedback from Tom and the LEWG mailing list to address.
- PBrett requested that a revision of the paper also address the potential ABI impact if a new encoding were to be added to the IANA registry; WTF-8 for example.
- Corentin replied that the ABI point to be addressed is the size and underlying type of the enumeration and agreed to amend the paper.
- Steve noted that string literals in headers are subject to ODR IFNDR concerns if included in TUs that are compiled with different encoding options and then linked together; this concern presumably applies to querying the literal encoding as well.
- Tom asked Corentin if he had come across the [ICU converter explorer](#) while researching encodings that were not present in the IANA registry and provided a link to <https://icu4c-demos.unicode.org/icu-bin/convexp?s=IBM&s=IANA>; this facility can identify encodings that are known to ICU, but are not present in the IANA registry.
- Corentin replied that he had heard of it and generally understands that ICU has support for many encodings.
- Tom pointed out that the ICU converter explorer also reveals aliases that are not present in the IANA registry.
- Corentin replied that he is uncertain how important support for such aliases is.

- Zach asked if the paper has already been reviewed by LEWG.
- Corentin replied that LEWG has initiated a mailing list review, but that there has not been much feedback regarding desire for the feature yet.
- Tom noted that Victor expressed strong support for it on the mailing list.
- [P2093R2: Formatted output](#):
 - Victor presented:
 - The goal is to integrate `std::format` directly with I/O in order to provide an alternative to `std::cout`, `std::printf`, and `std::fputs` that uses the `std::format` syntax.
 - The naming and interface are inspired by other languages.
 - The ability is provided to print via C streams (e.g., `stdout`) or C++ streams (e.g., `std::cout`).
 - There are multiple options for printing a newline; the most flexible option is to allow the programmer to specify the newline sequence themselves.
 - C++ iostreams doesn't work as expected with UTF-8 now, especially when printing to the Windows onsole.
 - On Windows, the run-time locale and terminal character set are distinct and separately managed in practice.
 - The reference implementation works portably on Windows and POSIX systems and does not require compilation with the Visual C++ `/utf-8` option.
 - The reference implementation has the performance benefits noted in [P0645](#) and outperforms both `std::printf` and `std::cout`.
 - The output that is produced is locale independent by default.
 - There is no global or shared formatting state to manage.
 - The binary size of calls is reduced through use of type erasure; `std::printf` without error handling is slightly smaller than the proposed `std::print`.
 - Jens asked if synchronization with C streams was disabled via `std::ios_base::sync_with_stdio()` when testing performance.
 - Victor confirmed that it was.
 - Jens asked why the performance results are an order of magnitude worse on Windows.
 - Victor replied that testing was on a different machine running in a VM.
 - Corentin expressed strong support for the feature.
 - Zach echoed that support.
 - Zach stated that he found the proposed `std::println` a little odd. From a LEWG perspective, the newline handling is unclear for `std::print(stdout, ...)` vs `std::print(cout, ...)`. From an SG16 perspective, it is unclear what a newline is at all.
 - Zach suggested leaving it to the programmer to specify the kind of newline they want.
 - Tom noted that the streams themselves are typically involved in formatting newlines and asked Zach if stream determined newline sequences addresses his concern.

- Zach replied that it does not and that he would prefer to change the default behavior for `std::print` to be to write to `std::stdout` and to remove `std::println`.
- Corentin professed support for `std::println` as a means to avoid concerns over what kind of newline sequence to use.
- Steve noted that differences between `std::println` and `std::print` will be encountered when programmers are learning these interfaces; just as the difference between printing of a newline differs for `std::puts` VS `std::fputs`.
- Steve expressed uncertainty regarding any additional value provided by `std::println`.
- Jens stated that there are two orthogonal features in the proposal that he would prefer to see separated if possible.
 - 1: Use of the `std::format` machinery to avoid need for temporary strings as would be returned by calls to `std::format`.
 - 2: `std::print` is encoding aware in a way that `std::format` is not.
- Jens noted that this feature appears to be focussed on terminals and that use of terminals is on the decline.
- Victor disagreed that `std::print` is a terminal facility since it can also be used to write to a file; in which case, it will still use the right encoding.
- Victor also disagreed that use of terminals is on a decline.
- Jens clarified that he does not wish to see these feature dropped from the proposal, but rather separated; though it isn't necessarily clear how a separated transcoding facility would work.
- Jens expressed surprise regarding use of the C `FILE*` streams over `std::streambuf`.
- Steve replied that locale information is associated with `std::streambuf` and that it manipulates data in the stream.
- Tom confirmed that the `std::codecvt` facet is attached via `std::streambuf`.
- Jens observed that that is a broken design.
- Steve agreed and noted that the saving grace is that most programmers avoid doing terrible things.
- PBrett expressed concern over handling of locales. In the `std::cout << std::format(...)` example, adding `L` to a format specifier will enable localized output using the global locale. Doing similarly with `std::print` would, by default, use `std::stdout` and the global locale rather than `std::cout` and its associated locale.
- PBrett expressed support for use of `std::print` with an explicit stream such that it uses the locale imbued in the stream and for dropping support for a default stream.
- Victor pointed out that the `std::format` and `std::print` examples are consistent; the global locale is used in each case.
- Victor noted that it is possible to pass a `std::ostream` to `std::print` and have its locale be used; LEWG has already expressed support for that approach.
- Victor added that `std::cout` has global state and therefore all the same problems as use of a global locale.
- PBrett stated that, if `std::print` is specified to use a default stream, that he would prefer it to use the locale imbued in `std::cout` rather than the global locale.

- PBrett noted that having to pass or query a `std::ostream` would bring back performance concerns, but that is also the desired behavior, so the performance improvements as presented are not particularly compelling.
- Jens noted that having synchronous I/O disabled will cause interleaving issues, so that does not reflect normal practice and distorts the performance improvements as presented.
- Victor noted that it is pretty uncommon to use `std::cout` for locale related purposes and stated that new tutorials for `std::print` could teach correct usage.
- Victor expressed a preference to not sacrifice performance for corner cases.
- Jens disagreed that this represents a corner case.
- Corentin requested an interface that allows just passing a locale.
- Tom stated that our next regularly scheduled telecon would be the week of Thanksgiving in the US, so we'll skip that week and the next telecon will be held in four weeks on December 9th. Similarly, the telecon after that one will be skipped due to Christmas leaving us meeting again on January 13th.